Impulse Generator Physics Engine — Technical Specification

Oct 10, 2026 · @DIya

1. Purpose and scope

The physics engine takes a target impulse (type, crest voltage, front and tail time) and a test setup, and returns a ranked list of buildable generator settings: number of stages, charging voltage per stage, front resistance and tail resistance, each with its predicted waveform and compliance status.

It is fully deterministic: the same inputs always give the same outputs. It models an ideal lumped circuit using formulas and numerical solutions only. It does not learn from data; the ML residual layer (Stage 2) sits on top of it and is out of scope here.

In scope

- Lightning impulse 1.2/50 µs and switching impulse 250/2500 µs
- Equivalent single-stage circuit of a multi-stage Marx generator
- Inverse calculation of ideal resistances, snapping to a resistor inventory, forward verification of the waveform
- Feasibility checks against generator limits and compliance checks against tolerance windows
- Exhaustive search and ranking over stage count and resistor choices

Out of scope

- Learning from measured or synthetic data
- Stage-level effects inside the generator (individual spark-gap timing, non-uniform stages)
- Corona, frequency-dependent losses, distributed (travelling-wave) behaviour of leads

Two fidelity modes are defined throughout this spec:

Mode	Purpose	Forward model
Calculator	Reproduce the official Hybrid Calculator sheet exactly (benchmark)	Closed-form approximations with constants 1.67, 2.5, 0.693
Exact	Correct solution of the same ideal circuit	Analytic double exponential (L = 0) or ODE (L > 0), with IEC 60060-1 parameter extraction

2. Inputs, outputs and units

Target times are fixed by the impulse type; resistances, stage count and charging voltage are calculated. All internal arithmetic is in SI units; conversion happens only at the input and output boundary.

Inputs

Input	Symbol	Display unit	SI unit	Sample value
Impulse type	—	Lightning / Switching	—	Lightning
Required test (crest) voltage	V_test	kV	V	1425
Test object capacitance	C_load	pF	F	850
Divider capacitance	C_div	pF	F	500
Stray capacitance	C_stray	pF	F	150
Stray loop inductance	L	µH	H	18.5
Expected efficiency (calculator mode only)	η	—	—	0.82
Front resistor inventory	—	Ω list + units per stage	Ω	10–500 Ω, 4 units
Tail resistor inventory	—	Ω list + units per stage	Ω	5–200 Ω, 4 units

Targets and tolerances (derived from impulse type)

Impulse	Front / peak target	Tail target	Front tolerance	Tail tolerance (calculator)	Tail tolerance (IEC)	Crest tolerance
Lightning	T1 = 1.2 µs	T2 = 50 µs	±30%	±20%	±20%	±3%
Switching	Tp = 250 µs	T2 = 2500 µs	±20%	±20%	±60%	±3%

Outputs (per ranked candidate)

- Settings: n, U0 per stage, front R per stage and total, tail R per stage and total, resistor recipe (e.g. 10 + (15 ∥ 200) Ω)
- Ideal (unsnapped) R values, for comparison
- Predicted waveform: T1 or Tp, T2, V_crest, η (exact mode)
- Percentage deviation from each target, normalised deviation, compliant yes/no
- Feasibility flags and score
- Run-level summary: count of candidates rejected per reason

3. Circuit model and governing equations

During discharge, n stages fire in series and the generator reduces to one equivalent stage: C1 discharges through R1 and L into the load C2, with R2 across C1.

Equivalent circuit values

$$C_2 = C_{load} + C_{div} + C_{stray}$$

$$C_1 = \frac{C_{stage}}{n}, \qquad V_0 = n\,U_0, \qquad R_1 = n\,R_{f,stage}, \qquad R_2 = n\,R_{t,stage}$$

C_bo (generator basic load capacitance, 545 pF on the Haefely datasheet) belongs in C2 for a complete model; the calculator sheet omits it, so it is a config option, off in calculator mode.

Stage count and charging voltage (calculator mode)

$$n_{start} = \left\lceil \frac{V_{test}}{U_{max}\,\eta} \right\rceil, \qquad U_0 = \frac{V_{test}}{n\,\eta}$$

Closed-form waveform relations (calculator mode)

$$T_1 = 1.67\sqrt{(R_1 C_2)^2 + 2.5\,L\,C_2}$$

$$T_2 = 0.693\,R_2\,(C_1 + C_2)$$

0.693 = ln 2: the tail is treated as a single exponential with time constant R2(C1 + C2), and T2 is the time to fall to 50%. The 1.67 and 2.5 are the sheet's empirical constants. The same T1 relation is used for switching time-to-peak.

Inverse relations (calculator mode)

$$R_1 = \frac{\sqrt{(T_1/1.67)^2 - 2.5\,L\,C_2}}{C_2}, \qquad R_2 = \frac{T_2}{0.693\,(C_1 + C_2)}$$

Exact model, L = 0: double exponential

$$v_2(t) = \frac{V_0}{R_1 C_2(\beta - \alpha)}\left(e^{-\alpha t} - e^{-\beta t}\right)$$

where α and β are the roots of $s^2 + a\,s + b = 0$, with

$$a = \frac{1}{R_1 C_1} + \frac{1}{R_1 C_2} + \frac{1}{R_2 C_1}, \qquad b =$$

```
\frac{1}{R_1 R_2 C_1 C_2}
```

Exact model, L > 0: state equations (states v1 across C1, loop current i, v2 across C2; initial state v1 = V0, i = 0, v2 = 0)

```
C_1\frac{dv_1}{dt} = -\frac{v_1}{R_2} - i, \qquad L\frac{di}{dt} = v_1 - R_1 i - v_2, \qquad C_2\frac{dv_2}{dt} = i
```

In exact mode, T1, Tp and T2 are not computed from formulas; they are extracted from v2(t) per IEC 60060-1 (Module D). The topology above (R2 on the generator side) must be confirmed against the generator manual.

4. System architecture

The engine is an inverse → snap → forward-verify pipeline wrapped in an exhaustive search: the inverse step aims, the snap step makes the result buildable, and the forward step measures what snapping cost.

[embedded content: physics engine architecture · 6 modules, one search loop]

Module A feeds the loop; for each stage count n, B computes ideal resistances, C proposes buildable values, D predicts the waveform for each, and E filters and scores it. Every scored candidate is kept and ranked.

Suggested code layout

```
physics/
  config.py     # A
  inverse.py    # B
  snap.py       # C
  forward.py    # D (calculator + exact)
  waveform.py   # D (IEC parameter extraction)
  scoring.py    # E
  search.py     # F
tests/
```

Build order: A → D (calculator) → B + round-trip test → C (round5) → E → F + benchmark test → C (combo) → D (exact) and IEC extraction. Stop at the benchmark test and confirm a full match before adding improvements.

5. Module A — Configuration and inputs

Module A holds every constant and input in three separate structures, so a generator can be swapped without touching the physics code.

GeneratorConfig

Field	Workbook generator	Haefely 508085/E datasheet
C_stage	3 µF	0.125 µF
n_min / n_max	2 / 15	2 / 12
U_max per stage	200 kV	200 kV
Energy limit per stage	none (disabled)	2.5 kJ
Total energy limit	none	30 kJ
C_bo	0 (omitted)	545 pF
Front inventory	10, 20, 30, 50, 75, 100, 150, 200, 300, 500 Ω	30 Ω, 465 Ω, 3.7 kΩ
Tail inventory	5, 10, 15, 20, 25, 30, 40, 50, 75, 100, 150, 200 Ω	520 Ω (LI), 22 kΩ (SI)
Max units per stage	4	1 (to confirm)
Constants k_front, k_LC, k_tail	1.67, 2.5, 0.693	same
Snap mode	round5	combo

TestSetup: impulse type, V_test, C_load, C_div, C_stray, L, η.

Target: T_front, T_tail and tolerance set, looked up from impulse type and mode (Section 2).

Rules

- Convert to SI on construction; store only SI values. Every field name carries no unit ambiguity (e.g. `c_load_F`, `l_H`).
- Validate: capacitances and L ≥ 0; V_test within 50–2400 kV; 0 < η ≤ 1; inventories non-empty and positive.
- Constants are config fields, never literals inside formulas.
- The energy limit is optional (`None` disables the check) so the workbook generator remains usable.

6. Module B — Inverse solver

For one stage count n, Module B returns the ideal continuous resistances that hit the target times exactly.

Interface: `ideal_resistances(setup, config, target, n) → IdealResult`

Field	Meaning
U0	Charging voltage per stage, V_test / (n·η)
C1, C2	Equivalent capacitances
R1_total, R2_total	Ideal total front and tail resistance

Field	Meaning
R1_stage, R2_stage	Totals divided by n
flags	e.g. `inductance_limited_front`, `overvoltage`

Algorithm (calculator mode)

1. Compute C2, C1 = C_stage / n, U0 = V_test / (n·η).
2. If U0 > U_max, set `overvoltage` (Module F skips this n).
3. Compute D = (T_front / k_front)² − k_LC·L·C2. If D ≤ 0, set `inductance_limited_front`: the inductance alone makes the front slower than target, so no resistance can reach it. Do not clamp to ~0 Ω as the sheet does.
4. R1_total = √D / C2; R2_total = T_tail / (k_tail·(C1 + C2)).
5. Divide by n for per-stage values.

Algorithm (exact mode): no closed-form inverse exists. Find R1 and R2 by root-finding on the forward model (`scipy.optimize.brentq` on T(R) − target), starting from the calculator-mode values as the bracket centre. Because T1 depends mainly on R1 and T2 mainly on R2, solve alternately (R1 with R2 fixed, then R2 with R1 fixed) until both change by less than 0.01%.

Property that must hold: feeding the unsnapped R1_total and R2_total into Module D in the same mode returns exactly T_front and T_tail (round-trip test, Section 11).

7. Module C — Snapper (resistor inventory)

Module C turns an ideal per-stage resistance into a short list of buildable candidates; it does not pick the winner.

Interface: `candidates(ideal_ohm, achievable_set, k) → [Candidate(value_ohm, recipe, units)]`

Snap modes

Mode	Achievable values	Use
round5	round(ideal / 5) × 5, inventory ignored	Benchmark: matches the calculator sheet
single	Single inventory resistors	Simple real-world mode
combo	Series, parallel and a + (b ∥ c) combinations, up to max units per stage	Best accuracy

Building the achievable set (once per inventory, at start-up)

1. For every multiset of 1 to max_units inventory values: add the series sum and the parallel value 1 / Σ(1/R).
2. For every value a and every pair (b, c): add a + b·c / (b + c).
3. Round each value to 0.001 Ω before de-duplicating; for each value keep the recipe with the fewest units.
4. Check that no recipe uses more units of one value than the inventory holds.

5. Store as a sorted array for binary search (`bisect`).

With the workbook inventory and 4 units, this yields about 940 tail values and 820 front values.

Selecting candidates: return the k nearest achievable values to the ideal (k = 3–5), always including the nearest below and the nearest above. Every candidate goes to Module D; the winner is chosen by the forward result, not by distance in ohms.

Sample run: ideal tail 23.94 Ω → round5 gives 25 Ω (+4.4%); combo gives 10 + (15 ∥ 200) = 23.95 Ω (+0.05%). Ideal front 49.52 Ω → round5 gives 50 Ω; combo also offers 30 + (20 ∥ 500) = 49.23 Ω.

8. Module D — Forward evaluator

Module D predicts the waveform of a fully specified circuit and is the source of truth for every decision the engine makes.

Interface: `evaluate(n, R1_total, R2_total, C1, C2, L, U0, mode) → Waveform(T_front, T_tail, V_crest, eta, overshoot, v_t?)`

Calculator mode

- T_front = k_front · $\sqrt{((R1 \cdot C2)^2 + k_LC \cdot L \cdot C2)}$
- T_tail = k_tail · R2 · (C1 + C2)
- V_crest = n · U0 · η (equals V_test by construction; η is an input)

Exact mode, step 1: compute v2(t)

Case	Method	Notes
L = 0	Analytic double exponential (Section 3)	Instant and exact
L > 0	`scipy.integrate.solve_ivp` on the 3-state system	Method `Radau` or `LSODA` (stiff: front and tail time constants differ ~1000×); `max_step` ≤ T_front / 200; `rtol` 1e-9, `atol` 1e-12; simulate to ≥ 3 × T_tail

Run with U0 = 1 V per stage. Crest voltage is linear in U0, so η = V_peak / n, and the required charging voltage follows as U0 = V_test / (n·η). Feasibility (U0 ≤ U_max) is checked afterwards in Module E.

Exact mode, step 2: IEC 60060-1 parameter extraction

Lightning impulse:

1. Find the peak (V_p, t_p).
2. On the rising edge, interpolate t30 and t90 where v = 0.3·V_p and 0.9·V_p.
3. T1 = 1.67 × (t90 − t30); virtual origin O1 = t30 − 0.3·T1.
4. On the tail, interpolate t50 where v = 0.5·V_p; T2 = t50 − O1.
5. Overshoot (only when L > 0 produces oscillation): relative overshoot evaluated with the test-voltage function; limit 10%.

Switching impulse:

1. Time to peak Tp = K · T_AB, where T_AB = t90 − t30 and K = 2.42 − 3.08×10⁻³·T_AB + 1.51×10⁻⁴·T2 (times in µs). Verify this expression against the standard text before use.
2. T2 is measured from the true origin (t = 0) to the 50% point on the tail.

Use `numpy.interp` on monotonic segments only (rising edge before the peak, falling tail after it).

Validation of D itself: the ODE with L → 0 must agree with the analytic solution to within 0.1%; calculator mode must reproduce the benchmark values in Section 11.

9. Module E — Feasibility, compliance and scoring

Module E rejects candidates the generator cannot run, then scores the rest by their worst parameter relative to its tolerance.

Hard feasibility filters (reject and record the reason)

Check	Rule	Reason code
Stage range	n_min ≤ n ≤ n_max	`stages_out_of_range`
Stage voltage	U0 ≤ U_max	`overvoltage`
Stage energy (if limit set)	½·C_stage·U0² ≤ W_stage_max	`energy_exceeded`
Total energy (if limit set)	n × ½·C_stage·U0² ≤ W_total_max	`total_energy_exceeded`
Inventory	recipe units within stock	`inventory_exceeded`
Front reachable	no inductance-limited flag from B	`inductance_limited_front`
Overshoot (exact mode)	≤ 10%	`overshoot_exceeded`

Compliance, per parameter p (front, tail, crest)

$$d_p = \frac{x_p^{pred} - x_p^{target}}{x_p^{target}}, \qquad e_p = \frac{|d_p|}{tol_p}, \qquad \text{compliant} \iff \max_p e_p \le 1$$

Score (lower is better)

$$S = \max_p e_p + w_1 \cdot units + w_2 \cdot n + w_3 \cdot \frac{U_0}{U_{max}}$$

The maximum, not the sum, is used because a test fails if any single parameter is out of tolerance. Weights w1–w3 are tie-breakers only; start with w1 = 0.002, w2 = 0.001, w3 = 0.01 so they never outweigh a 1% accuracy difference.

Sample run (calculator mode, round5): e_front = 0.8 / 30 = 0.027, e_tail = 4.4 / 20 = 0.22, e_crest = 0 → max = 0.22, compliant.

10. Module F — Search orchestrator and output record

Module F runs an exhaustive search over stage count and snapped resistor pairs; at roughly 175 forward evaluations per run, no optimisation algorithm is needed and the best option is guaranteed to be found.

Interface: `recommend(setup, config, mode, top_k=10) → RunResult(candidates, rejections)`

Loop

```
n_start = max(n_min, ceil(V_test / (U_max · η)))
for n in n_start .. n_max:
    ideal = B(setup, config, target, n)
    if ideal has a blocking flag: rejections[flag] += 1; continue
    for rf in C(ideal.R1_stage, front_set):
        for rt in C(ideal.R2_stage, tail_set):
            wave    = D(n, rf·n, rt·n, C1, C2, L, U0, mode)
            verdict = E(n, U0, rf, rt, wave, target)
            if not verdict.feasible: rejections[verdict.reason] += 1;
continue
            results.append(record(n, ideal, rf, rt, wave, verdict))
sort results by score; return top_k + rejections
```

In calculator mode with round5 snapping, the loop is restricted to n = n_start and one candidate per resistor, which reproduces the sheet exactly.

Output record (one per candidate)

Group	Fields
Settings	n, U0, R_front_stage, R_tail_stage, R1_total, R2_total, front recipe, tail recipe, units used
Ideal values	R1_stage_ideal, R2_stage_ideal
Waveform	T_front, T_tail, V_crest, η, overshoot; v(t) samples in exact mode for plotting
Compliance	deviation % and normalised deviation per parameter, compliant flag
Ranking	score, rank
Provenance	mode, snap mode, config name, engine version

Every record is kept, not only the winner: the dashboard uses it to explain the recommendation and plot target vs predicted waveforms, and Stage 2 uses the physics predictions as the baseline the ML residual model corrects.

11. Verification and test plan

The engine has matched the official sheet when calculator mode with round5 snapping reproduces every value below to a relative tolerance of 1e-6.

Benchmark inputs: load 850 pF, divider 500 pF, stray 150 pF, L = 18.5 µH, η = 0.82, workbook generator (3 µF, 15 stages, 200 kV). Lightning at 1425 kV; switching at 1050 kV.

Quantity	Lightning	Switching
C2	1500 pF	1500 pF
n	9	7
U0 per stage	193.0894 kV	182.9268 kV
C1	0.33333 µF	0.42857 µF
R1 total (ideal)	445.6993 Ω	99,800.24 Ω
R1 per stage (ideal / selected)	49.5221 / 50 Ω	14,257.18 / 14,255 Ω
R2 total (ideal)	215.4806 Ω	8,388.15 Ω
R2 per stage (ideal / selected)	23.9423 / 25 Ω	1,198.31 / 1,200 Ω
Predicted front / peak time	1.21003 µs	249.962 µs
Predicted tail time	52.2089 µs	2503.53 µs
Predicted crest	1425 kV	1050 kV

The lightning column is the cached output of the Hybrid Calculator sheet; the switching column is the same formulas evaluated with switching inputs.

Test suite (`pytest`, floats compared with `pytest.approx`)

1. **Units**: 1500 pF stored as 1.5e-9 F; display round-trip is lossless.
2. **Round trip**: B then D without snapping returns the target times exactly, both modes, both impulse types, for n = 2..n_max.
3. **Benchmark**: the table above, both impulse types.
4. **Snapper**: combo mode returns 23.953 Ω (10 + 15∥200) as nearest to 23.94 Ω; round5 returns 25 Ω.
5. **Analytic vs ODE**: agree within 0.1% for L = 1 nH.
6. **Exact-mode reference**: for n = 9, R1 = 450 Ω, R2 = 225 Ω on the workbook generator, exact mode gives T1 ≈ 1.82 µs and T2 ≈ 56.2 µs (cross-check value from an independent simulation).
7. **Edge cases**: inductance-limited front is flagged, not clamped; V_test at 50 kV and 2400 kV; no feasible n returns an empty list with rejection reasons.
8. **Search dominance**: in any mode, the top-ranked score is ≤ the score of the sheet's configuration.

12. Known limitations, open questions and references

The largest known limitation is the calculator-mode front formula: for the benchmark circuit it predicts T1 = 1.21 µs where the exact solution of the same ideal circuit gives about 1.82 µs, and for switching about 250 µs against about 496 µs.

Limitations

- Calculator mode treats crest voltage as an assumption (V_crest = V_test) and η as an input; only exact mode computes η from the circuit.
- The calculator T2 formula omits R1 from the discharge path; Kuffel's form $0.7 \cdot (R1 + R2)(C1 + C2)$ includes it.

- The switching time-to-peak in calculator mode reuses the lightning T1 relation, which IEC does not define that way.
- Lumped single-stage model: no stage-to-stage timing, travelling-wave or corona effects. These are left to the Stage 2 residual model.
- Resolution of the snapper beyond the forward model's accuracy adds no real value; combo snapping matters only in exact mode.

Open questions

- Which generator does the official workbook represent? Its 3 µF stages and 25 Ω / 1200 Ω tail resistors differ from the Haefely 508085/E datasheet (0.125 µF, 520 Ω / 22 kΩ, 2.5 kJ per stage).
- Is the tail resistor on the generator side or the load side of R1 in the actual generator? The exact-mode equations in Section 3 assume the generator side.
- Values of the external front resistors R_se1–R_se3 and whether series/parallel stacking is allowed on the real generator.
- Exact IEC 60060-1:2010 expression for the switching-impulse K factor (Section 8) to be confirmed from the standard text.

References

- E. Kuffel, W. S. Zaengl, J. Kuffel, *High Voltage Engineering: Fundamentals*, 2nd ed., Ch. 2 (impulse generator circuits)
- W. Hauschild, E. Lemke, *High-Voltage Test and Measuring Techniques*, chapter on impulse voltage generation
- IEC 60060-1:2010, *High-voltage test techniques — Part 1: General definitions and test requirements*
- IEC 60071-1, *Insulation co-ordination — Part 1: Definitions, principles and rules*
- SciPy documentation: `scipy.integrate.solve_ivp`, `scipy.optimize.brentq`
- Official inputs: HV IG Problem Statement (PDF) and Hybrid_Physics_ML_Impulse_Generator_Optimiser.xlsx, Hybrid Calculator sheet